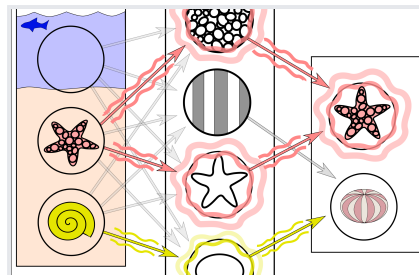
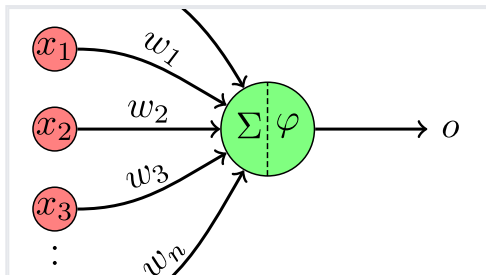
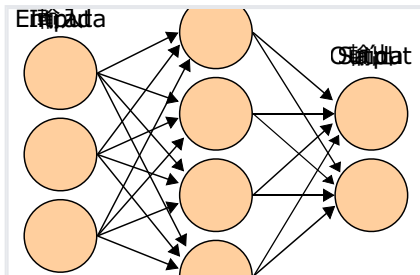


DNN: Deep Neural Network

깊은 완전연결 신경망의 핵심 이론

DNN은 입력 feature vector를 여러 층의 비선형 변환에 통과시켜, 데이터 안의 복잡한 관계를 함수로 근사하는 모델입니다.



학습 흐름

DNN을 읽는 순서

1. 입력 feature가 어떤 벡터가 되는지 본다.
2. 뉴런과 layer가 어떤 계산을 하는지 본다.
3. activation이 왜 필요한지 본다.
4. loss와 gradient가 어떻게 학습을 움직이는지 본다.
5. 과적합을 줄이고 검증 성능을 높이는 방법을 본다.

단원	핵심 질문
구조	왜 여러 층을 쌓는가?
학습	loss를 어떻게 줄이는가?
안정화	왜 정규화와 초기화가 필요한가?
일반화	훈련 성능이 실제 성능으로 이어지는가?

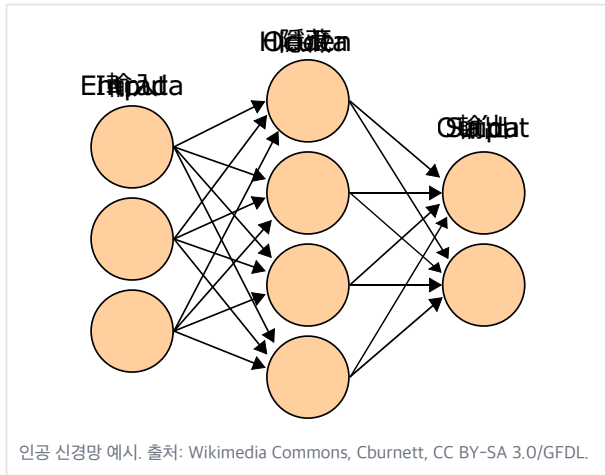
DNN은 “층을 많이 쌓은 모델”이 아니라, 입력 공간을 단계적으로 바꾸며 예측하기 쉬운 표현을 만드는 모델입니다.

DNN을 한 문장으로

여러 개의 함수 합성

- 한 층은 affine transform과 activation으로 구성됩니다.
- 깊은 네트워크는 이런 층을 여러 번 합성한 함수입니다.
- 각 층은 원본 feature를 다른 표현 공간으로 변환합니다.
- 마지막 층은 문제에 맞는 출력 형태로 바꿉니다.

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \phi(z^{(\ell)})$$



모델 전체는 여러 층의 함수를 차례로 합성한 하나의 큰 함수로 이해할 수 있습니다.

선형 모델에서 DNN으로

선형 모델의 한계

- 선형 회귀와 로지스틱 회귀는 입력 feature의 가중합을 기반으로 판단합니다.
- feature 사이의 복잡한 상호작용은 사람이 직접 만들어 넣어야 합니다.
- 예를 들어 집값은 면적 하나가 아니라 위치, 연식, 학군, 교통 접근성의 조합으로 결정됩니다.

DNN의 접근

관점	선형 모델	DNN
표현	원본 feature 그대로	hidden representation 학습
경계	직선/평면 중심	비선형 decision boundary
상호작용	사람이 설계	모델이 일부 학습
위험	해석은 쉬움	과적합 관리 필요

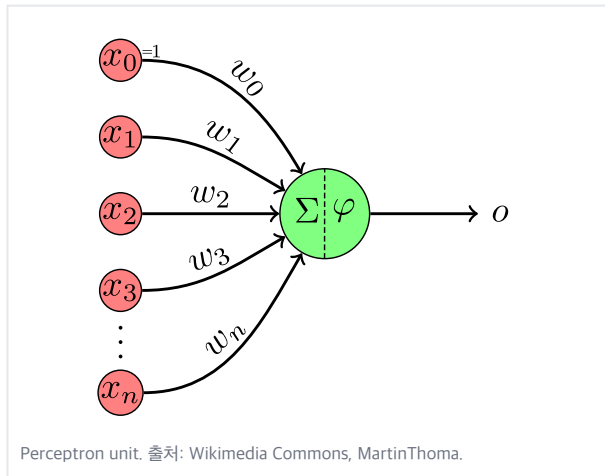
$$a = \phi(w^{\top}x + b)$$

Neuron

가장 작은 계산 단위

- 뉴런은 입력 vector와 weight의 내적을 계산합니다.
- bias는 decision boundary의 위치를 이동시키는 역할을 합니다.
- activation function은 계산 결과에 비선형성을 부여합니다.
- 뉴런 하나는 작은 함수이고, layer는 뉴런들을 병렬로 묶은 함수입니다.

$$a = \phi(w^{\top} x + b)$$



weight는 “무엇을 중요하게 볼지”, bias는 “판단 기준을 어디에 둘지”를 조절합니다.

Layer

행렬 연산으로 묶으면 빠르다

- 실제 DNN은 뉴런을 하나씩 계산하지 않고 행렬 연산으로 처리합니다.
- 입력 batch가 B개, feature가 d개이면 입력은 보통 B×d 행렬입니다.
- GPU는 큰 행렬 곱을 매우 빠르게 처리하므로 DNN 학습과 추론에 적합합니다.

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \phi(z^{(\ell)})$$

W는 weight matrix, b는 bias vector입니다.

activation phi를 통과해야 여러 층을 쌓는 의미가 생깁니다.

Depth

깊이는 표현 변환의 횟수다

- 얕은 모델은 원본 feature에서 바로 정답을 예측합니다.
- 깊은 모델은 입력을 여러 중간 표현으로 바꾼 뒤 예측합니다.
- 앞쪽 layer는 비교적 단순한 조합을 만들고, 뒤쪽 layer는 더 추상적인 조합을 만듭니다.
- 깊이가 늘면 표현력은 커지지만 학습 난이도와 과적합 위험도 커집니다.

단계	표 데이터 예	이미지 예
입력	면적, 방 수, 위치	픽셀
낮은 표현	크기/거리 조합	edge, 색 변화
높은 표현	입지·규모 패턴	부품, 질감
출력	가격, 등급	클래스

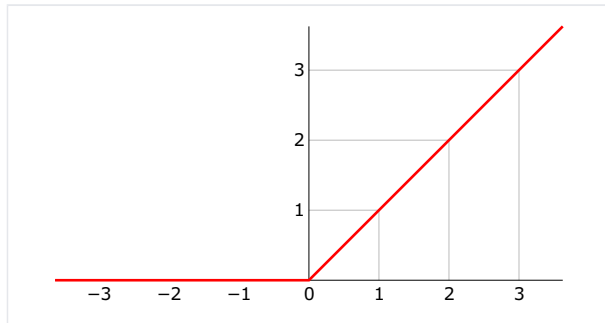
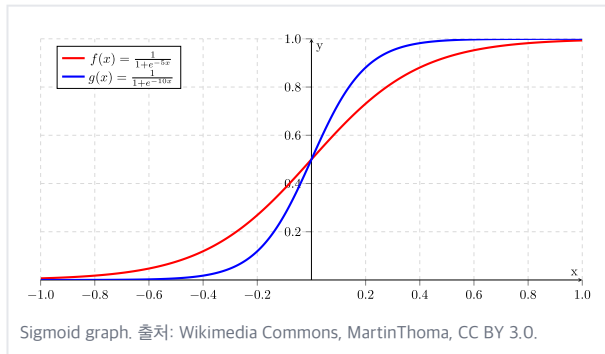
$$P_{\text{MLP}} = \sum_{\ell=1}^L (d_{\ell-1} + 1) d_{\ell}$$

Activation Function

비선형성이 없으면 깊어도 선형이다

- activation이 없으면 여러 affine layer를 쌓아도 결국 하나의 affine transform과 같습니다.
- Sigmoid와 tanh는 출력 범위가 제한되어 확률적 해석에는 좋지만, 깊은 네트워크에서는 gradient가 작아질 수 있습니다.
- ReLU는 단순하고 gradient 흐름이 좋아 DNN의 기본 선택지가 되었습니다.

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$



Output Layer

마지막 layer는 문제 형식에 맞춘다

- 회귀는 실수값을 직접 출력하는 linear output을 자주 씁니다.
- 이진 분류는 sigmoid로 양성 클래스 확률을 냅니다.
- 다중 분류는 softmax로 클래스별 확률분포를 만듭니다.
- 출력층 설계와 loss function은 반드시 서로 맞아야 합니다.

문제	출력	대표 loss
집값 예측	$\hat{y} \in \mathbb{R}$	MSE, MAE
불량/정상	$\hat{p} \in [0,1]$	Binary CE
숫자 0-9	$\hat{p} \in \mathbb{R}^{10}$	Cross Entropy
다중 라벨	label마다 확률	Binary CE

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}}, \quad \mathcal{L}_{CE}(y, \hat{p}) = - \sum_{k=1}^C y_k \log \hat{p}_k$$

Loss Function

학습이 줄이려는 숫자

- loss는 모델 출력이 정답과 얼마나 다른지 수치화합니다.
- 회귀에서는 예측값과 실제값의 거리, 분류에서는 정답 클래스의 확률을 봅니다.
- 좋은 loss는 문제의 비용 구조와 연결되어야 합니다.
- 훈련은 평균 loss를 줄이는 방향으로 parameter를 이동시키는 과정입니다.

$$\mathcal{L}_{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}}, \quad \mathcal{L}_{CE}(y, \hat{p}) = - \sum_{k=1}^C y_k \log \hat{p}_k$$

MSE는 회귀, cross entropy는 분류에서 가장 기본적인 출발점입니다.

Empirical Risk

데이터셋 위에서 평균 손실을 줄인다

- 모델이 직접 최소화하는 것은 실제 세계의 모든 오류가 아니라 학습 데이터의 평균 오류입니다.
- 학습 데이터가 실제 사용 환경과 다르면 loss를 잘 줄여도 배포 성능이 나빠질 수 있습니다.
- 그래서 데이터셋 분리와 검증 성능 관찰이 DNN에서도 핵심입니다.

$$\hat{R}_{\mathcal{D}}(f) = \frac{1}{|\mathcal{D}|} \sum_{(x_i, y_i) \in \mathcal{D}} \ell(f(x_i), y_i)$$

데이터 품질과 분포가 바뀌면 DNN이 최적화하는 목표 자체가 달라집니다.

DNN은 강력하지만, 결국 주어진 데이터셋에서 정의된 loss를 최적화합니다.

Backpropagation

각 parameter가 loss에 준 영향을 계산한다

- forward pass는 입력에서 출력과 loss를 계산합니다.
- backward pass는 chain rule로 각 parameter의 gradient를 계산합니다.
- optimizer는 gradient를 사용해 weight와 bias를 갱신합니다.
- backpropagation은 “각 층이 얼마만큼 책임이 있는지”를 효율적으로 나누는 알고리즘입니다.

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(a^{(\ell-1)} \right)^\top, \quad \delta^{(\ell)} = \frac{\partial \mathcal{L}}{\partial z^{(\ell)}}$$

delta는 해당 층의 pre-activation이 loss에 미치는 민감도를 뜻합니다.

Chain Rule

깊은 합성함수의 미분

- DNN은 여러 층의 합성함수입니다.
- 특정 weight가 loss에 미치는 영향은 그 weight 이후 모든 층을 거쳐 전달됩니다.
- chain rule은 이 영향을 layer별 local gradient의 곱으로 분해합니다.
- 자동미분 프레임워크는 계산 그래프를 저장한 뒤 역방향으로 gradient를 전파합니다.

$$\frac{\partial \mathcal{L}}{\partial a^{(1)}} = \left(\prod_{\ell=2}^L W^{(\ell)\top} D^{(\ell)} \right) \frac{\partial \mathcal{L}}{\partial z^{(L)}}$$

여러 작은 값이 계속 곱해지면 앞쪽 layer의 gradient가 작아지는 vanishing gradient가 생길 수 있습니다.

Gradient Descent

기울기의 반대 방향으로 움직인다

- learning rate η 는 한 번에 움직이는 보폭입니다.
- 너무 크면 loss가 불안정하게 튀고, 너무 작으면 학습이 매우 느립니다.
- SGD는 일부 mini-batch로 gradient를 추정합니다.
- Adam은 gradient의 평균과 분산을 함께 이용해 parameter별 보폭을 조절합니다.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

optimizer를 바꾸면 같은 network라도 학습 속도와 안정성이 크게 달라질 수 있습니다.

Mini-Batch, Epoch, Iteration

용어	의미	예시
sample	데이터 한 개	주택 한 건, 이미지 한 장
batch	한 번에 forward/backward하는 묶음	64개 sample
iteration	batch 하나로 parameter를 한 번 갱신	1 update
epoch	train set 전체를 한 번 사용	10,000개 데이터를 모두 봄

mini-batch는 계산 효율과 gradient 안정성 사이의 절충입니다. batch size가 커지면 GPU 효율은 좋아질 수 있지만, 메모리 사용량과 일반화 특성도 함께 바뀝니다.

Initialization

시작점이 나쁘면 학습이 막힌다

- 모든 weight를 0으로 시작하면 뉴런들이 같은 gradient를 받아 서로 다른 역할을 배우지 못합니다.
- 너무 큰 weight는 activation과 gradient를 폭발시킬 수 있습니다.
- 너무 작은 weight는 신호와 gradient를 사라지게 만들 수 있습니다.
- Xavier/Glorot, He 초기화는 layer의 fan-in/fan-out을 고려해 분산을 맞춥니다.

상황	흔한 증상	대응
weight가 너무 큼	loss가 NaN, 발산	learning rate 감소, 초기화 점검
weight가 너무 작음	학습이 거의 안 됨	He/Xavier 초기화
ReLU가 죽음	출력이 0에 고정	learning rate, bias, LeakyReLU 검토
layer가 깊음	앞쪽 gradient 약함	normalization, residual 구조

Normalization

입력과 중간 표현의 스케일을 안정화한다

- 표 데이터 feature는 단위가 서로 다릅니다. 면적, 속도, 거리처럼 크기가 다른 값은 scaling이 필요합니다.
- 이미지 pixel은 보통 0-1 또는 평균 0, 분산 1 근처로 정규화합니다.
- Batch Normalization은 mini-batch 통계를 이용해 layer 내부 activation의 분포를 안정화합니다.
- normalization은 optimization을 쉽게 만들지만, train/test 동작 차이를 이해해야 합니다.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

BN은 정규화한 뒤 학습 가능한 γ , β 로 다시 스케일과 위치를 조절합니다.

Regularization

훈련 데이터 암기를 줄인다

- DNN은 parameter가 많아 train loss를 매우 낮게 만들 수 있습니다.
- 중요한 목표는 train set 암기가 아니라 unseen data에서 낮은 오류를 내는 것입니다.
- L2 weight decay는 weight가 지나치게 커지는 것을 억제합니다.
- Dropout은 학습 중 일부 activation을 무작위로 꺼서 co-adaptation을 줄입니다.

$$\tilde{a}^{(\ell)} = \frac{m \odot a^{(\ell)}}{1 - p}, \quad m_i \sim \text{Bernoulli}(1 - p)$$

dropout은 보통 train mode에서만 적용하고, evaluation mode에서는 꺼야 합니다.

Overfitting 읽기

loss curve가 보내는 신호

- train loss는 계속 감소하지만 validation loss가 증가하면 과적합입니다.
- train loss도 validation loss도 높으면 모델 용량 부족, 학습 부족, feature 품질 문제를 의심합니다.
- validation 성능이 흔들리면 split 크기, class imbalance, random seed를 확인합니다.

관찰	가능한 원인	대응
train만 좋음	과적합	dropout, weight decay, 데이터 증가
둘 다 나쁨	underfitting	더 큰 모델, feature 개선, epoch 증가
loss가 튼	learning rate 과다	learning rate 감소
val만 불안정	검증셋 작음	stratified split, k-fold

좋은 DNN 실험은 accuracy 한 줄보다 train/validation curve를 함께 읽습니다.

Train / Validation / Test

DNN에서도 데이터 분리는 실험 설계다

- train set은 weight를 학습하는 데 사용합니다.
- validation set은 architecture, learning rate, epoch, regularization 선택에 사용합니다.
- test set은 모든 선택이 끝난 뒤 마지막 한 번의 성능 보고에 사용합니다.
- test 성능을 보며 모델을 계속 고치면 test set도 사실상 validation set이 됩니다.

split	DNN에서 하는 일
train	forward/backward, parameter update
validation	early stopping, hyperparameter 선택
test	최종 일반화 성능 추정

$$\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset, \quad \hat{R}_{\text{test}}(f) = \frac{1}{m} \sum_{j=1}^m \ell(f(x_j), y_j)$$

집값 예측 DNN

feature vector 설계

집값 예측 DNN의 입력은 주택 또는 지역 블록을 설명하는 수치 vector입니다.

feature	처리 예
면적, 방 수, 연식	표준화
지역, 건물 유형	one-hot 또는 embedding
위도/경도	스케일링, 거리 feature 추가
소득, 인구, 학군	결측치·이상값 처리

label과 출력

항목	설계
label	실제 거래가 또는 중간 주택 가격
output	실수 1개
loss	MSE, MAE, Huber
metric	RMSE, MAE, MAPE

$$\mathcal{L}_{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

집값 예측에서 중요한 점

표 데이터 DNN의 실전 포인트

- feature scale을 맞추지 않으면 일부 feature가 gradient를 지배할 수 있습니다.
- 범주형 feature는 one-hot이면 차원이 커지고, embedding이면 학습이 필요합니다.
- 가격 분포가 long-tail이면 target에 log transform을 적용하는 경우가 많습니다.
- 지역 정보는 강력하지만, leakage와 편향을 함께 점검해야 합니다.

선택	장점	주의
log price 예측	극단값 영향 완화	해석 시 역변환 필요
embedding	지역/유형 표현 학습	데이터 적으면 불안정
feature crossing	상호작용 반영	차원 증가
early stopping	과적합 방지	validation split 품질 중요

표 데이터에서는 DNN 구조보다 feature 정의, 스케일링, 검증 설계가 성능을 더 크게 좌우할 때가 많습니다.

MNIST 분류 DNN



MNIST 손글씨 숫자 예시. 출처: Wikimedia Commons, Josef Steppan, CC BY-SA 4.0.

이미지도 vector로 펼치면 MLP 입력이 된다

- 28×28 grayscale 이미지는 784차원 vector로 펼칠 수 있습니다.
- 출력층은 10개 숫자 클래스의 softmax 확률을 냅니다.
- 단순 MLP도 MNIST에서는 동작하지만, 이미지의 2차원 구조를 버린다는 한계가 있습니다.
- CNN은 이 한계를 보완하기 위해 local pattern과 parameter sharing을 사용합니다.

Hyperparameter Checklist

항목	의미	시작점
hidden layers	표현 변환 단계 수	2-4층
hidden units	layer별 폭	64, 128, 256
learning rate	갱신 보폭	Adam 1e-3 근처
batch size	한 번에 처리할 sample 수	32-256
dropout	activation 무작위 제거	0.1-0.5
weight decay	weight 크기 패널티	1e-5-1e-3

좋은 실험은 한 번에 모든 값을 바꾸지 않고, validation metric을 기준으로 좁은 범위부터 체계적으로 조정합니다.

DNN Troubleshooting

먼저 확인할 것

- loss가 감소하는지 확인합니다.
- train/validation metric을 분리해서 봅니다.
- 입력 feature의 결측, scale, 이상값을 확인합니다.
- label이 잘못 연결되었거나 shuffle 과정에서 어긋나지 않았는지 확인합니다.

증상	우선 점검
accuracy가 랜덤 수준	label mapping, output/loss 조합
loss가 NaN	learning rate, normalization, 이상값
train은 좋고 val은 나쁨	과적합, leakage 반대 방향 split
재현이 안 됨	seed, 데이터 순서, nondeterministic 연산
추론이 느림	layer width, batch inference, quantization

DNN의 강점과 한계

강점

- 비선형 관계와 feature interaction을 자동으로 학습할 수 있습니다.
- 이미지, 텍스트, 음성의 다른 모델과 결합하기 쉽습니다.
- 충분한 데이터와 계산 자원이 있으면 큰 표현력을 가집니다.

한계

- 표 데이터에서는 tree ensemble보다 항상 우수하지 않습니다.
- parameter가 많아 과적합과 tuning 비용이 큼니다.
- 입력 분포 변화와 leakage에 민감합니다.
- 해석 가능성이 낮아 설명과 검증 절차가 중요합니다.

DNN은 만능 해법이 아니라, 데이터 규모·feature 구조·검증 목적이 맞을 때 강력한 선택지입니다.

핵심 요약

구조

행렬 변환과 activation을 여러 번
합성해 표현을 바꿉니다.

학습

loss의 gradient를
backpropagation으로 계산하고
optimizer가 parameter를 갱신합
니다.

안정화

초기화, normalization, learning
rate가 gradient 흐름을 좌우합니
다.

일반화

validation curve, regularization,
early stopping으로 과적합을 관리
합니다.

데이터

feature scale, label 품질, split 설
계가 DNN 성능의 바닥을 결정합니
다.

연결

CNN은 DNN 원리를 이미지 구조
에 맞게 바꾼 대표적인 특수 구조입
니다.

참고자료 및 이미지 출처

- Goodfellow, Bengio, Courville, Deep Learning: <https://www.deeplearningbook.org/>
- Nielsen, Neural Networks and Deep Learning: <http://neuralnetworksanddeeplearning.com/>
- scikit-learn California Housing dataset: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html
- TensorFlow Datasets MNIST catalog: <https://www.tensorflow.org/datasets/catalog/mnist>
- Wikimedia Commons, Artificial neural network.svg, Cburnett, CC BY-SA 3.0/GFDL: https://commons.wikimedia.org/wiki/File:Artificial_neural_network.svg
- Wikimedia Commons, Perceptron-unit.svg, MartinThoma: <https://commons.wikimedia.org/wiki/File:Perceptron-unit.svg>
- Wikimedia Commons, Sigmoid-function.svg, MartinThoma, CC BY 3.0: <https://commons.wikimedia.org/wiki/File:Sigmoid-function.svg>
- Wikimedia Commons, Ramp function.svg, Qef, Public Domain: https://commons.wikimedia.org/wiki/File:Ramp_function.svg
- Wikimedia Commons, Simplified neural network example.png, Mikael Haggstrom, CC0: https://commons.wikimedia.org/wiki/File:Simplified_neural_network_example.png
- Wikimedia Commons, MnistExamples.png, Josef Steppan, CC BY-SA 4.0: <https://commons.wikimedia.org/wiki/File:MnistExamples.png>